ServiceNow Project Report: Script-Controlled Access Control Lists (ACLs)

Team ID : SWTID-2026-5765 **Team Size : 4**

Team Leader : YUVARAJ B
Team member : DHARANIDHARAN T
Team member : ATCHAYA A
Team member : CHANDRU S

Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Platform: ServiceNow System Administration

Target Audience: ServiceNow Developers, System Administrators, Security Engineers

Executive Summary

Data security and access governance form the cornerstone of enterprise service management. In standard ServiceNow configurations, Role-Based Access Control (RBAC) handles macro-level permissions (e.g., granting full Read or Write access to all records in a custom table based on assigned roles). However, modern enterprise requirements demand dynamic, context-aware security rules where access permissions must adapt based on the state or specific field values of an individual record.

This project implements **Script-Controlled Access Control Lists (ACLs)** to enforce conditional record-level security across custom and core tables in ServiceNow. By leveraging Server-Side JavaScript (current and gs objects), custom roles, and specific CRUD operations, the project restricts record visibility and editability dynamically. This comprehensive report details the complete end-to-end architecture across all 7 project milestones.

Table of Contents

1. Milestone 1: Creation of Users and Security Roles
2. Milestone 2: Custom Tables & Schema Architecture
3. Milestone 3: Implementation of Read Access Control Lists (ACL-READ)
4. Milestone 4: Implementation of Create Access Control Lists (ACL-CREATE)
5. Milestone 5: Implementation of Write Access Control Lists (ACL-WRITE)
6. Milestone 6: Implementation of Delete Access Control Lists (ACL-DELETE)
7. Milestone 7: Verification, Security Audit & Project Conclusion

8. Appendix A: Script Snippets & Code Documentation
9. Appendix B: Best Practices & Maintenance Guidelines

Milestone 1: Creation of Users and Security Roles

Overview & Objectives

The foundation of ServiceNow security relies on the User-Group-Role hierarchy. To properly evaluate and test script-controlled access constraints, dedicated security roles and test user personas were created within the ServiceNow instance.

System Configuration Steps

1. **Role Definition:**
    - **Role Name:** x_custom_app.restricted_user
    - **Description:** Standard user role required for basic record interactions under conditional constraints.
    - **Role Name:** x_custom_app.sec_admin
    - **Description:** Elevated role granted rights to manage, override, or audit record access.
2. **User Creation & Provisioning:**
    - **Persona 1 (Standard User - Restricted):** Assigned x_custom_app.restricted_user.
    - **Persona 2 (Department Manager):** Assigned manager attributes for dynamic manager-based visibility rules.
    - **Persona 3 (System Auditor):** Unassigned custom roles to test positive negative-testing scenarios (Access Denied validation).
3. **Group Configuration:**
    - Created **Security Operations Group** (sec_ops) and assigned custom roles at the group level to ensure alignment with ServiceNow security governance standards (never assign roles directly to users in production environments).

Milestone 2: Custom Tables & Schema Architecture

Overview & Objectives

To demonstrate dynamic field-value security evaluation, a custom target table (x_custom_app_sensitive_record) was designed with specific state and categorization fields.

Data Model & Field Specifications

Field Label	Column Name	Data Type	Purpose / Access Control Relevance
Number	number	String (Auto-Number)	Unique identifier for the record.
Short Description	short_description	String	Brief summary of the record.
Classification	u_classification	Choice	Values: Public, Internal, Confidential, Restricted. Controls dynamic READ permissions.
State	state	Choice / Integer	Values: Draft (1), In Review (2), Approved (3), Closed/Locked (4). Controls WRITE permissions.
Assigned To	assigned_to	Reference (sys_user)	Used in script conditions (current.assigned_to == gs.getUserID()).
Approval Manager	u_approval_manager	Reference (sys_user)	Used for hierarchical access validation.

Milestone 3: Implementation of Read Access Control Lists (ACL-READ)

Overview & Objectives

Configure read ACL rules to ensure users can only view records based on the u_classification field and the user's explicit role or relationship to the record.

Security Logic Execution

- **Rule Type:** Record ACL
- **Operation:** read
- **Name:** x_custom_app_sensitive_record.* and x_custom_app_sensitive_record
- **Role Requirement:** x_custom_app.restricted_user

Script Implementation Logic

JavaScript

```
(function executeRule(current, previous /*null when calling query*/) {
```

```
  // Grant access if the record is Public or Internal
  if (current.u_classification == 'public' || current.u_classification == 'internal') {
    answer = true;
    return;
  }

  // Dynamic checks for Confidential / Restricted records
  var currentUser = gs.getUserID();
  if (current.assigned_to == currentUser || current.u_approval_manager == currentUser)
{
    answer = true;
  } else {
    answer = gs.hasRole('x_custom_app.sec_admin');
  }
})(current, previous);
```

Milestone 4: Implementation of Create Access Control Lists (ACL-CREATE)

Overview & Objectives

Restricts who can insert new records into the custom table, ensuring that mandatory initial fields (such as default classification) are initialized within proper operational limits.

Configuration Details

- **Operation:** create
- **Target:** x_custom_app_sensitive_record
- **Conditions:** Requires the user to hold active membership in authorized operational groups alongside the base role.

Milestone 5: Implementation of Write Access Control Lists (ACL-WRITE)

Overview & Objectives

Implement state-dependent editing controls. Once a record transitions to Closed/Locked (4), field editing must be restricted via server-side ACL evaluation, regardless of UI policy or Client Script bypass attempts.

Script Implementation Logic

JavaScript

```
(function executeRule(current, previous) {
  // Lock record editing if state is Closed/Locked (4)
  if (current.state == 4) {
    // Only allow System Admin/Sec Admin to override locked state
    answer = gs.hasRole('x_custom_app.sec_admin');
  } else {
    // Allow write if assigned to user or manager
    answer = (current.assigned_to == gs.getUserID() ||
gs.hasRole('x_custom_app.restricted_user'));
  }
})(current, previous);
```

Milestone 6: Implementation of Delete Access Control Lists (ACL-DELETE)

Overview & Objectives

Prevents unauthorized or accidental deletion of critical records. Deletion rights are tightly controlled via script checks and restricted strictly to administrators or record creators within a 24-hour grace window.

Configuration Details

- **Operation:** delete
- **Target:** x_custom_app_sensitive_record
- **Rule:** If record state is past Draft state, deletion is completely disabled for non-admin users.

Milestone 7: Verification, Security Audit & Project Conclusion

Execution & Testing Matrix

Test Case	Persona / User	Scenario Description	Expected Result	Actual Result
TC-01	Standard User	Open Public classification record	Read Access Granted	**PASS**

Test Case	Persona / User	Scenario Description	Expected Result	Actual Result
TC-02	Standard User	Open Restricted classification record (Not Assigned)	Read Access Denied	**PASS**
TC-03	Assigned User	Edit record in In Review state	Write Access Granted	**PASS**
TC-04	Assigned User	Edit record in Closed/Locked state	Write Access Denied	**PASS**
TC-05	Standard User	Attempt record deletion on active record	Delete Action Hidden/Denied	**PASS**

Project Summary

Through the execution of Milestones 1 through 7, a robust security model was established using script-controlled Access Control Lists in ServiceNow. By moving beyond rigid role-based rules into dynamic field-level script evaluations, the platform ensures strict governance, prevents data leakage, and enforces record lifecycle locks effectively.

Appendix A: Best Practices & Maintenance Guidelines

1. **Performance Optimization:** Keep script evaluations lightweight. Avoid executing heavy GlideRecord queries inside ACL scripts whenever possible, as ACLs execute repeatedly for every row returned in list views.
2. **Debugging Tooling:** Utilize Debug Security Rules (ACL Debugger) in ServiceNow developer instances to trace rule evaluation order and boolean outcomes (true/false).
3. **Execution Order:** Remember the ServiceNow ACL evaluation hierarchy: **Role Check** →**Condition Check** →**Script Check**. If the Role or Condition fails, the Script will not execute.